

Ingenic®

Halley6 Platform Application Development Manual

Author: System Software Department

Version: 2.0

Date: 2022.12.5



北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.



Halley6 Platform Application Development Manual

Copyright © Ingenic Semiconductor Co. Ltd 2021. All rights reserved.

Release history

Date	Revision	Change
2021.10.26	1.0	First release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

Ingenic Semiconductor Co., Ltd.

Ingenic Headquarters, East Bldg. 14, Courtyard #10
Xibeiwang East Road, Haidian District, Beijing, China,
Tel: 86-10-56345000
Fax:86-10-56345001
Http: [//www.ingenic.com](http://www.ingenic.com)

ingenic

CATALOG

1 Introduction.....	5
1.1 Introduction to Directory Structure.....	5
1.2 Special attention.....	5
2 Ingenic platform compilation configuration and command introduction	
.....	6
2.1 Ingenic platform default configuration.....	6
2.1.1 Default configuration in buildroot.....	6
2.1.2 Custom configuration in buildroot.....	6
2.2 Ingenic vendor configuration in buildroot.....	7
2.2.1 system config for rootfs Configuration.....	7
2.2.2 Wifi configure.....	8
2.2.3 pretest configure.....	9
2.2.4 Bluetooth configure.....	10
2.2.5 h264e netlink server support configure.....	11
2.2.6 Applypatch configure.....	11
2.3 compile command.....	12
2.3.1 Compilation Rules.....	12
2.3.2 Compile the generated files.....	13
2.3.3 Method of compiling modules individually.....	13
2.3.4 The method of compiling the buildroot package separately.....	14
2.3.5 The method of compiling after the module content is changed.....	14
3 Ingenic Platform Application Introduction.....	15
3.1 cimutils.....	15
3.1.1 Function Introduction.....	15
3.1.2 How to use.....	15
3.1.2.1 Module compilation.....	15
3.1.2.2 Module usage.....	16
4 Add and remove applications to the target file system.....	17
4.1 halley6_base.mk introduce.....	17
4.2 rootfs-overlay Folder introduction.....	17
5 Import and run the application to the development board.....	18
5.1 Import the application using the adb tool.....	18
5.1.1 Configuration adb tool.....	18
5.1.2 Adb tool usage.....	18
5.2 Importing applications using sshd.....	19

5.2.1 Configuring the sshd tool.....	19
5.2.2 Using the sshd tool.....	19
5.3 Importing applications using telnetd.....	20
5.3.1 Telnetd Instructions.....	21
5.4 Importing applications using tftp.....	21
5.4.1 Configure tftp tool.....	21
5.4.2 Use of tftp tool.....	22
5.5 Importing applications using lrzsz.....	23
5.5.1 Configuring the lrzsz tool.....	23
5.5.2 Using the lrzsz tool (minicom).....	23
6 Introduction to Platform Compilation Rules (General).....	25
6.1 Creating a Platform Application.....	25
6.1.1 Compiling core macros.....	25
6.1.2 Commonly used compiler interface macros.....	25
6.1.3 Compiling the example.....	26
6.2 Importing third-party applications.....	27
6.2.1 Third-party application core macros.....	27
6.2.2 Common interface macros of third-party applications.....	27
6.2.3 Compiling the example.....	28
6.3 Add a module for copying files.....	28
6.3.1 Compiling core macros.....	29
6.3.2 Common interface macros.....	29
6.3.3 Compiling the example.....	29
6.4 Automatically generate modules by command.....	30
6.4.1 Example of generating a copy module.....	31
6.5 File system overlay rootfs-overlay.....	32
6.6 Making the root file system.....	32
6.6.1 jffs2 file system creation.....	32
6.6.2 ubifs file system creation.....	32
6.6.3 ext4 file system creation.....	33
7 Apps debug.....	34
7.1 Debug using the gdb command line tool.....	34
7.1.1 gdb tool configuration.....	34
7.1.2 Gdb local debugging.....	34
7.1.3 gdbserver + gdb Remote debugging.....	35
7.1.3.1 Installation of gdbserver.....	35
7.1.3.2 gdbserver + gdb Debug method.....	36
7.1.4 gdb Debug using the command.....	38
7.2 Debug using the vscode integrated development environment.....	39
7.2.1 vscode installation and use.....	39
7.2.2 Debugging Environment Construction and Project Compilation.....	40

7.2.3 Start debugging.....	40
----------------------------	----

ingenic

Key words	document format
abstract	This document describes the basic format of the document .
abbreviation	
References	

1 Introduction

Halley6 software platform is a set of linux system release and development platform developed by Ingenic. The platform conducts guided compilation through the Build.mk file. The following functions are supported.

- Support compiling c files, cpp files.
- Support the compilation of dynamic library and static library.
- Support the compilation of executable bin files.
- Support for adding third-party applications and libraries.
- Supports separate compilation of modules.

1.1 Introduction to Directory Structure

The directory file structure of the platform is shown in the following figure.

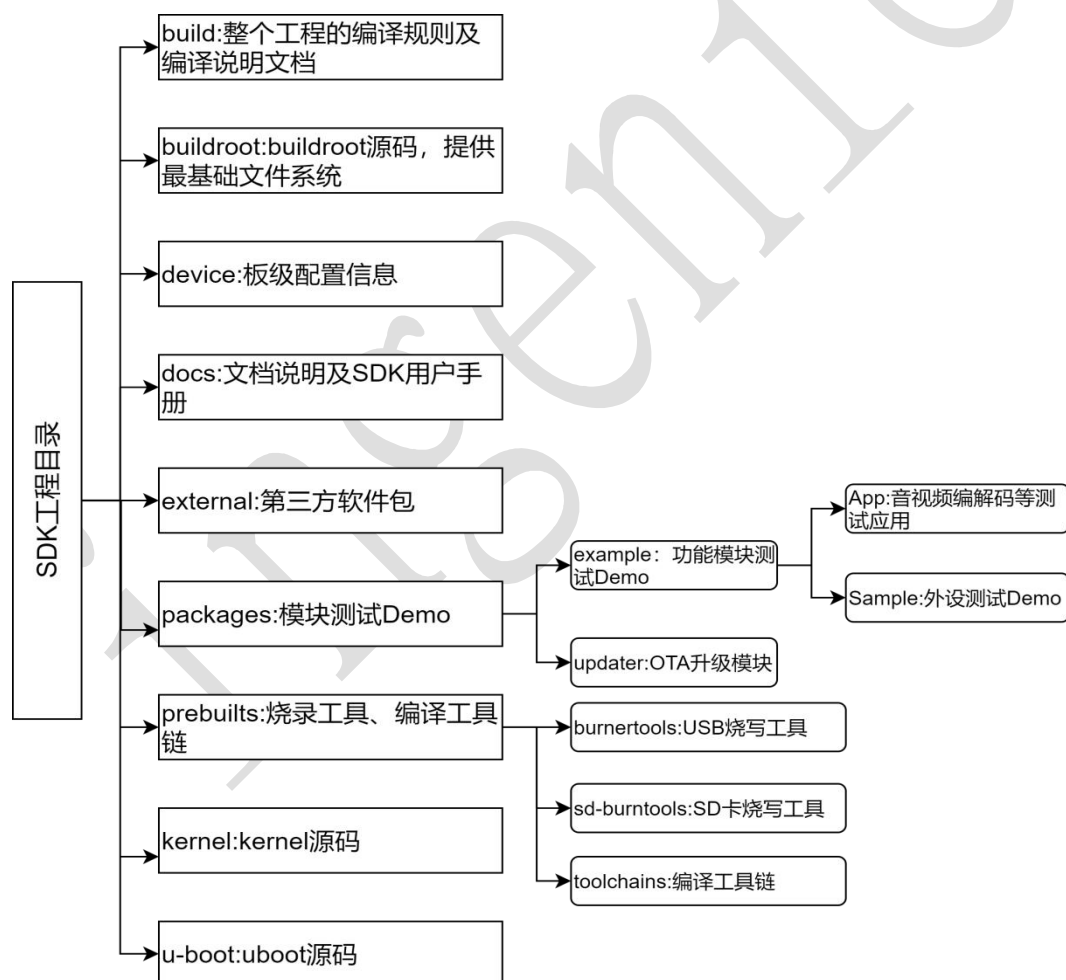


Figure 1-1 Halley6 platform directory structure

1.2 Special attention

All contents of this document are described by taking the halley6 v1.0 development board under

the X1600 platform as an example. Developers can refer to the code they actually downloaded.

2 Ingenic platform compilation configuration and command introduction

The compilation configuration of the platform is mainly explained by taking the configuration in buildroot as an example.

2.1 Ingenic platform default configuration

2.1.1 Default configuration in buildroot

The halley6 software platform provides several default configurations for different storage media. The default configurations are stored in the buildroot/configs/ folder. The following is the default configuration of several different storage media corresponding to the board level of halley6.

```
halley6_linux_awtk_defconfig  
halley6_linux_defconfig  
halley6_linux_lvgl_defconfig  
halley6_msc_ota_defconfig  
halley6_nor_defconfig  
halley6_ramdisk_defconfig
```

2.1.2 Custom configuration in buildroot

The default configuration in buildroot provides some common tools and settings, which cannot meet the needs of all users. Users can customize the configuration according to their own needs.

For example, to add a ffmpeg tool to buildroot. The operation is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Press the "/" key to enter the search mode, and enter ffmpeg in the search box. The search results are shown in the figure below.

```

Search Results

Symbol: BR2_PACKAGE_FFMPEG [=n]
Type : bool
Prompt: ffmpeg
Location:
  -> Target packages
(1) -> Audio and video applications
    Defined at package/ffmpeg/Config.in:18
    Depends on: BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    Selected by [n]:
    - BR2_PACKAGE_MPV [=n] && BR2_TOOLCHAIN_HAS_THREADS [=y] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y] && BR2_TOOLCHAIN_GCC_AT_LEAST_4_5 [=y] && (BR2_TOOLCHAIN_HAS_ATOMIC [=y] |
    - BR2_PACKAGE_OMXPLAYER [=n] && BR2_ARM [=n] && BR2_USE_MMU [=y] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y] && BR2_INSTALL_LIBSTDCPP [=n] && BR2_TOOLCHAIN_HAS_THREADS [=y] &&
    - BR2_PACKAGE_TQ10 [=n] && BR2_USE_MMU [=y] && BR2_TOOLCHAIN_HAS_THREADS [=y] && BR2_INSTALL_LIBSTDCPP [=n] && BR2_USE_WCHAR [=n] && (BR2_PACKAGE_PYTHON [=n] || BR2_PACKAG
    - BR2_PACKAGE_GST1_LIBAV [=n] && BR2_PACKAGE_GSTREAMER1 [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_MPD_FFMPEG [=n] && BR2_PACKAGE_MPD [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_SQUEEZELITE_FFMPEG [=n] && BR2_PACKAGE_SQUEEZELITE [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_QTSWENGINE [=n] && BR2_PACKAGE_QTS [=n] && BR2_PACKAGE_QTSWENGINE_ARCH_SUPPORTS [=n] && BR2_TOOLCHAIN_USES_GLIBC [=n] && BR2_HOST_GCC_AT_LEAST_4_9 [=y] &&
    - BR2_PACKAGE_OPENCV_WITH_FFMPEG [=n] && BR2_PACKAGE_OPENCV [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_OPENCV3_WITH_FFMPEG [=n] && BR2_PACKAGE_OPENCV3 [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_OPENCV4_WITH_FFMPEG [=n] && BR2_PACKAGE_OPENCV4 [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_MINIDLNA [=n] && !BR2_STATIC_LIBS [=n] && BR2_USE_WCHAR [=n] && BR2_USE_MMU [=y] && BR2_TOOLCHAIN_HAS_THREADS [=y] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_TVHEADEND_TRANSCODING [=n] && BR2_PACKAGE_TVHEADEND [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]

Symbol: BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
Type : bool
Defined at package/ffmpeg/Config.in:1
Depends on: !BR2_nios2 [=n] && !BR2_ARM_CPU_ARMV7M [=n] && !BR2_m68k_cf [=n] && (!BR2_or1k [=n] || BR2_TOOLCHAIN_GCC_AT_LEAST_9 [=n])

Symbol: BR2_PACKAGE_FFMPEG_AVRESAMPLE [=n]
Type : bool
Prompt: Build libavresample
Location:
  -> Target packages
(2) -> ffmpeg (BR2_PACKAGE_FFMPEG [=n])
    Defined at package/ffmpeg/Config.in:77
    Depends on: BR2_PACKAGE_FFMPEG [=n]
    Selected by [n]:
    - BR2_PACKAGE_OMXPLAYER [=n] && BR2_ARM [=n] && BR2_USE_MMU [=y] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y] && BR2_INSTALL_LIBSTDCPP [=n] && BR2_TOOLCHAIN_HAS_THREADS [=y] &&
    - BR2_PACKAGE_OPENCV3_WITH_FFMPEG [=n] && BR2_PACKAGE_OPENCV3 [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_OPENCV4_WITH_FFMPEG [=n] && BR2_PACKAGE_OPENCV4 [=n] && BR2_PACKAGE_FFMPEG_ARCH_SUPPORTS [=y]
    - BR2_PACKAGE_FREESWITCH [=n] && BR2_INSTALL_LIBSTDCPP [=n] && !BR2_STATIC_LIBS [=n] && BR2_TOOLCHAIN_HAS_THREADS [=y] && BR2_USE_MMU [=y] && BR2_USE_WCHAR [=n] && BR2_PACK
    ( 27% )
  
```

4) According to the Location prompt on the search results, go to Target packages/Audio and video applications/Configuration (or press 1), then check and enter ffmpeg, and select the functions you want to add according to your needs.

5) Save and exit

2.2 Ingenic vendor configuration in buildroot

Ingenic manufacturer configuration in buildroot refers to some configurations under Ingenic packages in buildroot configuration. The configurable content is shown in the following figure.

```

[*] system config for rootfs --->
[*] wifi --->
[*] pretest --->
[*] bluetooth --->
[*] h264e netlink server support
[*] applypatch
[ ] iqs
[ ] igst
[ ] libffplay (NEW)
  
```

Figure 2-1 Ingenic Vendor Configuration

2.2.1 system config for rootfs Configuration

In halley6 platform, some system configuration can be performed on system config for rootfs. It mainly includes the following configurations:

Table 2-1 Configurable items of rootfs of Ingenic manufacturers

Configurable	illustrate
--------------	------------

enable adb service set dns server cache dns mount usr data: mount_ubifs.sh userdata /usr/data/ mount usr data: mount mmcblk0p7 to /usr/data/ mount storage: mount mmcblk0p9 to /storage/ enable sd burn enable rootlogin when using open ssh enable usb composite gadgets	open adb service open dns service Mount the user data partition at boot Turn on the sd card burning function Log in to the system as an administrator
---	---

As shown below.

```

-- system config for rootfs
[ ] enable adb service
[ ] set dns server
-*- install mount_ubifs.sh
[ ] cache dns
[*] mount usr data: mount_ubifs.sh userdata /usr/data/
[ ] mount usr data: mount mmcblk0p7 to /usr/data/
[ ] mount storage: mount mmcblk0p9 to /storage/
[ ] enable sd burn
[*] enable rootlogin when using open ssh
[*] enable usb composite gadgets

```

Figure 2-2 The optional configuration content of rootfs in Ingenic manufacturer configuration

The specific configuration method is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/system config for rootfs/
- 4) Select the content to be configured
- 5) Save and exit

2.2.2 Wifi configure

In halley6 platform, WIFI can be configured. The following are several configurable contents of WIFI.

Table 2-2 Configurable items of Ingenic manufacturer's WIFI

Configurable	illustrate
common wifi device	Choose a common WIFI
select wifi chip	Select the model of the WIFI chip

wpa_supplicant.conf file path	Set wpa_supplicant.conf file path
hostapd.conf file path	Set the hostapd.conf file path
hostap interface	set hostap interface name
wifi auto startup	Configure whether WIFI starts automatically
wifi use ap mode	Configure whether to use hotspot mode for WIFI
wifi mac address	Set the mac address of WIFI
wifi mac address path	Configure the mac address file path

As shown below

```
-- wifi
[ ] common wifi device
    select wifi chip --->
-* wifi use wpa_supplicant
(/etc/wpa_supplicant.conf) wpa_supplicant.conf file path
(/etc/hostapd.conf) hostapd.conf file path
(wlan0) hostap interface
[*] wifi auto startup
[*] wifi use ap mode
(C8:93:46:45:C3:80) wifi mac address
(/data/misc/wifi/) wifi mac address path
```

Figure 2-3 wifi optional configuration items in Ingenic manufacturer configuration

The method of WIFI configuration is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/ wifi/
- 4) Select the item you want to configure
- 5) Save and exit

2.2.3 pretest configure

On the Halley6 platform, pretest can be configured to test onboard modules, and the functions that can be tested are as follows.

Table 2-3 Configurable items of Ingenic manufacturer's Bluetooth

configuration item	illustrate
bluetooth test	Bluetooth function test
wifi test	WIFI function test
network test	network function test
mmc test	mmc test, you can choose different test modes

amic test	amic test
dmic test	dmic test, you can choose the number of test dmics
speaker test	speaker test
camera test	Camera test, you can choose the number of cameras
lcd test	lcd display test
usb test	usb test, you can choose the mode host and device of the usb
touchscreen test	Touch screen function test

As shown below.

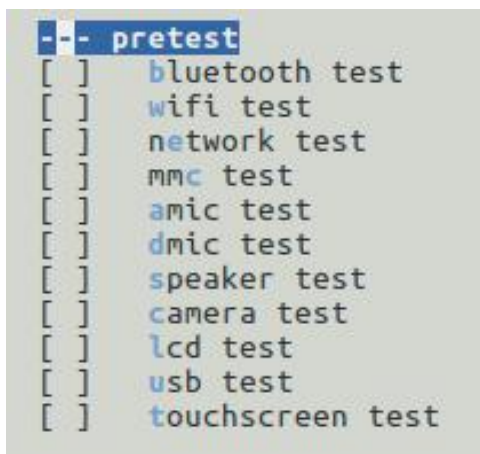


Figure 2-4 Optional configuration items of pretest in Ingenic manufacturer configuration

The way to configure pretest is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configure Ingenic packages/pretest, the path of the configuration options is shown in the figure below.
- 4) Select the configuration item to be configured
- 5) Save and exit

2.2.4 Bluetooth configure

In the Halley6 platform, you can configure whether Bluetooth supports BSA server, and select the Bluetooth chip used. The specific configuration method is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/bluetooth, The path of the configuration options is

shown in the following figure.

```
Symbol: BR2_PACKAGE_BLUETOOTH [=y]
Type : bool
Prompt: bluetooth
Location:
(1) -> Ingenic packages
Defined at package/ingenic/bluetooth/Config.in:1
```

Figure 2-5 The configuration path of bluetooth in Ingenic manufacturer configuration

2.2.5 h264e netlink server support configure

In halley6 platform, you can configure whether to support h264e netlink server. The specific configuration method is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/h264e netlink server support, The path to the configuration options is shown in the image below.

```
Symbol: BR2_PACKAGE_H264E_NL_SERVER [=y]
Type : bool
Prompt: h264e netlink server support
Location:
(1) -> Ingenic packages
Defined at package/ingenic/h264e-nl-server/Config.in:1
```

Figure 2-6 The configuration path of h264e netlink server support in Ingenic manufacturer configuration

2.2.6 Applypatch configure

In halley6 platform, you can configure whether to use applypatch. The specific configuration method is as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/applypatch, The path to the configuration options is shown in the image below.


```
Symbol: BR2_PACKAGE_APPLYPATCH [=y]
Type : bool
Prompt: applypatch
Location:
(1) -> Ingenic packages
Defined at package/ingenic/applypatch/Config.in:1
```

Figure 2-7 The configuration path of applypatch in Ingenic manufacturer configuration

2.3 compile command

2.3.1 Compilation Rules

Before compiling, first complete the initialization of the compilation toolchain and compilation configuration. Execute the following instructions to initialize the compilation toolchain.

```
source build/envsetup.sh
```

Then execute the following command to load the compiled configuration.

```
lunch
```

After the execution is complete, some optional configurations will be listed, as shown in the following figure.

```
You're building on Linux

Lunch menu... pick a combo:
 1. panda.v10_nand_4.4.94-eng
 2. panda.v10_nor_4.4.94-eng
 3. panda.v10_msc_4.4.94-eng
 4. panda.v10_nand_5.10-eng
 5. panda.v10_nor_5.10-eng
 6. panda.v10_msc_5.10-eng
 7. x1660evb.v10_nand_4.4.94-eng
 8. halley6.v10_nand_4.4.94-eng
 9. halley6.v10_nor_4.4.94-eng
10. halley6.v10_msc_4.4.94-eng
11. halley6.v10_msc_4.4.94_ota-eng
12. halley6.v10_nand_5.10-eng
13. halley6.v10_nor_5.10-eng
14. halley6.v10_msc_5.10-eng
```

For configuration selection, please refer to "Halley6 Software Platform Quick Development Guide".

The following are some common commands compiled for the platform. Enter the halley6 software platform **root directory** and execute the following commands to compile the corresponding content.

Table 2-4 halley6 platform compilation commands

compile command	illustrate
make	Overall compilation project
make buildroot	Compile buildroot separately
make buildroot-menuconfig	Configure buildroot
make buildroot-clean	Clear some files generated by buildroot compilation
make buildroot-distclean	Completely clear the files generated by buildroot compilation, including downloaded libraries
make buildroot-<packname>	Compile packages inside buildroot
make kernel	Compile the kernel separately
make kernel-menuconfig	Configure the kernel
make kernel-clean	Clear some files generated by kernel compilation
make kernel-distclean	Completely clear the files generated by kernel compilation
make uboot	Compile uboot separately
make uboot-menuconfig	configure uboot
make uboot-clean	Clear some files generated by uboot compilation
make uboot-distclean	Completely clear the files generated by uboot compilation
make (module naem)	Compile modules individually

2.3.2 Compile the generated files

After the compilation is complete, the out directory is generated with the following structure:

```

product
├── halley5           // board level
│   ├── image        // Generated object files (uboot, kernel, filesystem)
│   ├── obj           // Intermediate files and object files generated during
│   │   module compilation (before strip)
│   │   ├── sysroot   // file system (before strip)
│   │   └── system    // file system (including stripped modules)

```

After compiling buildroot alone, the generated rootfs will be stored in the obj/buildroot-intermediate/images/ directory. When the "make" command is executed for the overall compilation, the generated rootfs will be moved to the image directory.

2.3.3 Method of compiling modules individually

The compilation rules for a single module are as follows, executed in the top-level directory of the project:

```
make $(LOCAL_MODULE)
```

That is, make "module name" can compile a single module independently. Take the cimutils test module under `packages/example/App/cimutils` as an example, the module LOCAL_MODULE (in Build.mk) is assigned the value: cimutils, and the following command can be executed separately to compile cimutils module:

```
make cimutils
```

2.3.4 The method of compiling the buildroot package separately

In the halley6 platform's buildroot/packages/ directory, there are some optional packages. When you need to use these packages, you can compile them as follows.

```
make buildroot-<packname>
```

To compile ffmpeg under buildroot/packages/, execute the following command to compile.

```
make buildroot-ffmpeg
```

2.3.5 The method of compiling after the module content is changed

After the module is compiled, the content of the module is modified. Using the method of compiling the module separately may cause the changed content not to be updated to the final generated file system. The most effective way is to delete the intermediate files generated by the module compilation, and then recompile the module.

The following takes the cimutils module as an example to introduce the compilation method after the module content is changed.

1) After the module content is changed, the intermediate files in the out directory need to be deleted before compiling. The intermediate file path generated by cimutils is: **out/product/halley6/obj/DEPANNER/cimutils-intermediate**, delete the directory prefixed with cimutils.

2) Execute the instruction to compile the module separately to compile the module. ie: make cimutils.

3) Execute the make command to update the generated content to the system image under out/product/halley6/image/.

Note: The intermediate file paths generated by software packages under packages are generally in the out/product/halley6/obj/DEPANNER/ or out/product/halley6/obj/packages/example/App/ directory. The software package generation path under buildroot/ is generally in the out/product/halley6/obj/buildroot-intermediate/build/ directory.

3 Ingenic Platform Application Introduction

Ingenic platform applications refer to some applications developed by Ingenic itself. as shown in the table below.

Table 3-1 Application of Inzheng Platform

Platform application	illustrate
cimutils	Picture preview and camera interface

3.1 cimutils

3.1.1 Function Introduction

cimutils test cim camera image preview and take pictures, support image JPEG, raw and bmp format output.

- Source code path: packages/example/App/cimutils
- The software flow chart is as follows:

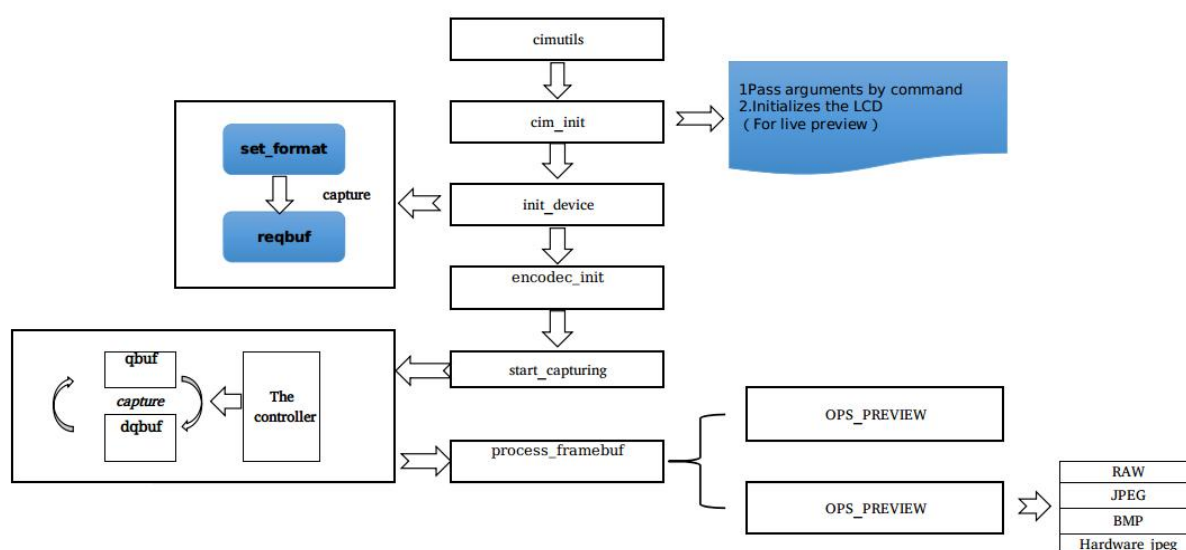


Figure 3-5 Flow chart of Cimutils software application on Ingenic platform

3.1.2 How to use

3.1.2.1 Module compilation

Enter the root directory of halley6 platform and execute the following command to compile the module.

```
make cimutils
make
```

After the module is compiled, the executable file `cimutils` is generated and stored in the `out/product/halley6/system/usr/bin/` directory.

3.1.2.2 Module usage

Execute the following instructions on the halley6 development board to run the module.

```
cimutils -C -I dev/video11 -t raw10bggr -f test.raw -x 640 -y 480 -m
```

parameter:

-C Generate pictures in JPEG or BMP format
-I Specify device node
-f Specify output file type
-t Specify the input format
-x Specify image width
-y Specify image height
-m output grayscale

Test Results:

Check if the `test.raw` file is displayed normally.

4 Add and remove applications to the target file system

4.1 halley6_base.mk introduce

halley6_base.mk is used to configure the modules compiled by the project. Located in the device/halley6 (board level)/ directory. The macro PRODUCT_MODULES is used to add or delete the modules compiled by the general project. Also in this file are default configurations for different file systems and storage media. These can be individually configured according to your needs.

4.2 rootfs-overlay Folder introduction

The rootfs-overlay folder is in the device/halley6 (board level)/ directory. The files in this folder will replace the files with the same name in the file system when the file system is compiled. Thereby you can configure your own file system individually.

5 Import and run the application to the development board

5.1 Import the application using the adb tool

5.1.1 Configuration adb tool

In the halley6 platform, you can directly configure and install the **adb** tool as follows:

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Ingenic packages/system config for rootfs/enable adb service, As shown

below.

```
There is no help available for this option.
Symbol: BR2_SYSTEM_CONFIG_ENABLE_ADB [=n]
Type : bool
Prompt: enable adb service
Location:
  -> Ingenic packages
    -> system config for rootfs (BR2_PACKAGE_SYSTEM_CONFIG [=y])
Defined at package/ingenic/system_config/Config.in:18
Depends on: BR2_PACKAGE_SYSTEM_CONFIG [=y]
```

Figure 5-1 The path to configure the adb tool in buildroot

Install the adb tool on the PC (take Ubuntu16.04 as an example):

The installation command is:

```
sudo apt-get install adb
```

After the installation is complete, run adb version to check the adb version. If the version information is displayed normally, the installation is successful.

5.1.2 Adb tool usage

- 1) Transfer files to the development board

```
adb push Local path (the path of the file to be transferred from the PC to the board) Target path (the path where the file is stored on the board)
```

- 2) Download the file from the board to the pc

```
adb pull Target path (the path of the file on the board) Local path (the path where the file is to be saved on the PC side)
```

3) Adb Common commands:

Table 5-1 Common commands of adb

adb Common commands	illustrate
adb help	View command help for adb
adb devices	View equipment
adb shell	into the terminal
adb kill-server	Out of service
adb start-server	Open service
adb push local path target path	Upload files to the device
adb pull target path local path	Download files from device to PC

5.2 Importing applications using sshd

5.2.1 Configuring the sshd tool

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Target packages/Networking applications/openssh,As shown below.

```

Symbol: BR2_PACKAGE_OPENSSSH [=y]
Type   : bool
Prompt: openssh
Location:
  -> Target packages
(1)   -> Networking applications
Defined at package/openssh/Config.in:1
Depends on: BR2_USE_MMU [=y]
Selects: BR2_PACKAGE_OPENSSL [=y] && BR2_PACKAGE_ZLIB [=y]
Selected by [n]:
- BR2_PACKAGE_SSHFS [=n] && BR2_USE_WCHAR [=y] && BR2_TOOLCHAIN_HAS
- BR2_PACKAGE_MOSH [=n] && BR2_PACKAGE_PROTOBUF_ARCH_SUPPORTS [=y]

```

Figure 5-2 The path to configure the sshd tool in buildroot

5.2.2 Using the sshd tool

Before using the sshd tool, please make sure that the board and the host to be communicated can ping each other.

The sshd (secure shell) service uses the ssh protocol to remotely open other host shell services. When in use, open a shell service of the board through the pc. The instructions for logging into the

board are as follows.

```
ssh -X remote host user@remote host ip
```

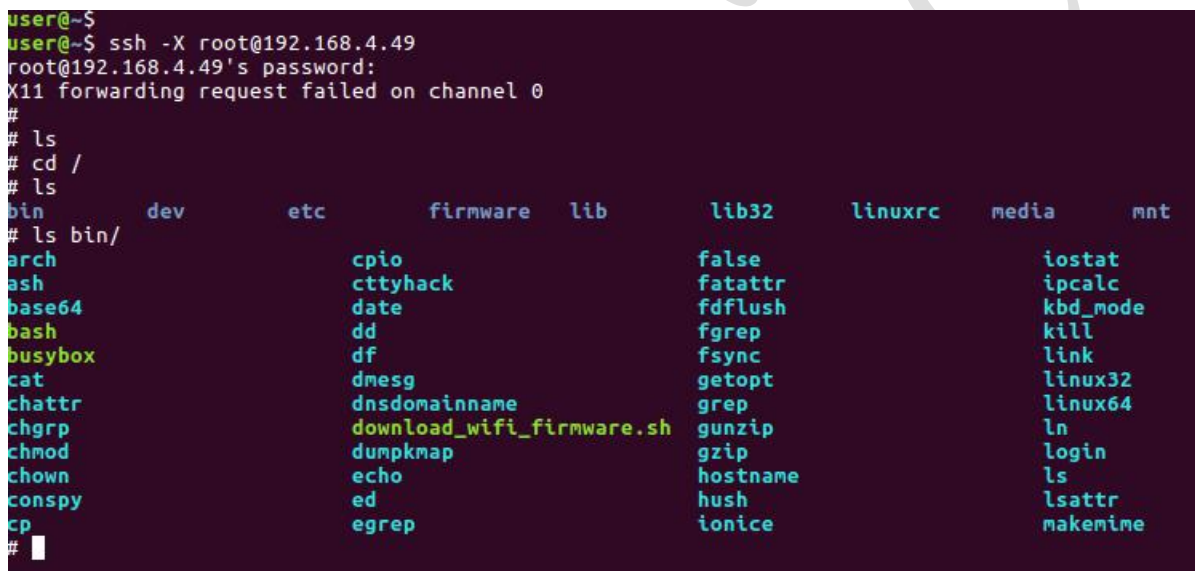
For example, the user name logged in on the board is root and the ip is 192.168.4.49. The login method is:

```
ssh -X root@192.168.4.49
```

The default login password is 123456, which can be set in buildroot as follows.

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration System configuration/Enable root login with password/Root password

The following figure shows the command operation for successful login.



```
user@~$ ssh -X root@192.168.4.49
root@192.168.4.49's password:
X11 forwarding request failed on channel 0
#
# ls
# cd /
# ls
bin          dev          etc          firmware    lib          lib32        linuxrc      media        mnt
# ls bin/
arch          cpio          cpio          cpio          cpio          cpio          cpio          cpio          cpio
ash           cpiohack     cpiohack     cpiohack     cpiohack     cpiohack     cpiohack     cpiohack     cpiohack
base64        date          date          date          date          date          date          date          date
bash          dd            dd            dd            dd            dd            dd            dd            dd
busybox       df            df            df            df            df            df            df            df
cat           dmesg        dmesg        dmesg        dmesg        dmesg        dmesg        dmesg        dmesg
chatrr        dnsdomainname dnsdomainname dnsdomainname dnsdomainname dnsdomainname dnsdomainname dnsdomainname dnsdomainname
chgrp         download_wifi_firmware.sh download_wifi_firmware.sh download_wifi_firmware.sh download_wifi_firmware.sh download_wifi_firmware.sh download_wifi_firmware.sh download_wifi_firmware.sh
chmod         dumpkmap     dumpkmap     dumpkmap     dumpkmap     dumpkmap     dumpkmap     dumpkmap     dumpkmap
chown         echo         echo         echo         echo         echo         echo         echo         echo
conspy        ed            ed            ed            ed            ed            ed            ed            ed
cp            egrep        egrep        egrep        egrep        egrep        egrep        egrep        egrep
#
```

Figure 5-3 Screenshot of successfully logging in to the board using the ssh tool

NOTE: If you cannot connect, and print

```
ssh: connect to host 192.168.4.109 port 22: Connection refused
```

Check whether the device node /dev/null permission is

```
# ls -l dev/null
crw-rw-rw- 1 root root 1, 3 Jan 1 00:00 dev/null
```

If not, you need to modify the device node /dev/null, the method is as follows

```
rm /dev/null
mknod /dev/null c 1 3
chmod 666 /dev/null
```

5.3 Importing applications using telnetd

The telnetd service has been added in the configuration file, no further configuration is required.

5.3.1 Telnetd Instructions

Before using the telnet tool, please make sure that the board and the host to be communicated can ping each other.

The telnet command is usually used to log in remotely. The method used is as follows.

```
telnet [parameter][host]
```

The optional parameters are as follows.

- 8 Allows the use of 8-bit character data, including input and output.*
- a Attempt to automatically log in to the remote system.*
- b <host alias> specify the remote host name using an alias*
- d Start debug mode.*
- E Filter out escape characters.*
- f The effect of this parameter is the same as specifying the "-F" parameter.*
- F When using Kerberos V5 authentication, adding this parameter can upload the authentication data of the local host to the remote host.*
- k <domain name> When using Kerberos authentication, add this parameter to let the remote host use the specified realm name instead of the host's domain name.*
- K Do not automatically log in to the remote host.*
- l <username> Specify the user name to log in to the remote host.*
- L Allows output of 8-bit character data.*
- n <record file> specifies a file to record related information.*
- r Use a user interface similar to the rlogin command.*
- x Assuming the host has a feature that supports data encryption, use it.*

For example, when the login user name is root and the ip is the host of 192.168.4.49, perform the following operations

```
telnet 192.168.4.49
```

Then follow the prompts to enter your username and password.

5.4 Importing applications using tftp

5.4.1 Configure tftp tool

1) Enter halley6 project

2) Execute make buildroot-menuconfig

3) Configuration Target package/Networking applications/dnsmasq/tftp support, The configuration is as follows:

```
Symbol: BR2_PACKAGE_DNSMASQ_TFTP [=y]
Type   : bool
Prompt: tftp support
Location:
  -> Target packages
  -> Networking applications
(2)    -> dnsmasq (BR2_PACKAGE_DNSMASQ [=y])
Defined at package/dnsmasq/Config.in:11
Depends on: BR2_PACKAGE_DNSMASQ [=y]
```

Figure 5-5 The path to configure the tftp tool in buildroot

5.4.2 Use of tftp tool

Before using the tftp tool, please make sure that the board and the host to be communicated can ping each other.

1) Download the file

The board is used as the client side, use the following command to download the file to the board

```
tftp -g -l destination filename -r source filename server address
```

If you want to download a file named A.txt from the server to the board, and rename the newly downloaded file to B.txt, use the following command.

```
tftp -g -l B.txt -r A.txt 192.168.4.50
```

2) Upload file

When uploading files from the board (client) to the server, use the following command.

```
tftp -p -r destination filename -l source filename server address
```

For example, upload the file C.txt from the board to the tftp root target of the Server, and change its name to D.txt using the following command.

```
tftp -p -r D.txt -l C.txt 192.168.4.50
```

3) Parameter description

-l	Abbreviation for local, followed by the name of the source file that exists in the Client, or the name of the file renamed after downloading the Client;
-r	The abbreviation of remote, followed by Server, that is, the source file name in the root directory of the PC tftp server, or the file name after uploading Server;
-g	Abbreviation for get, used when downloading files;
-p	Abbreviation for put, used when uploading files.

5.5 Importing applications using lrzsz

5.5.1 Configuring the lrzsz tool

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Target package/Networking applications/lrzsz, The configuration is as follows:

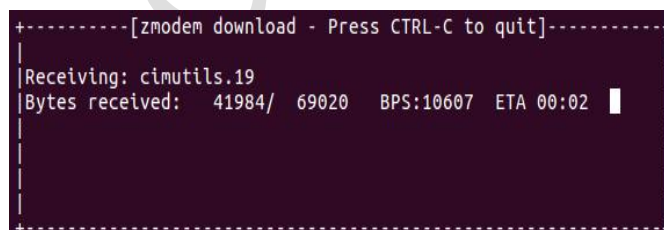
```
BR2_PACKAGE_LRZSZ:
Portable and fast implementation of the X/Y/Zmodem protocols.
http://www.ohse.de/uwe/software/lrzsz.html

Symbol: BR2_PACKAGE_LRZSZ [=y]
Type : bool
Prompt: lrzsz
Location:
-> Target packages
-> Networking applications
Defined at package/lrzsz/Config.in:1
Depends on: !BR2_STATIC_LIBS [=n]
```

Figure 5-6 The path to configure the lrzsz tool in buildroot

5.5.2 Using the lrzsz tool (minicom)

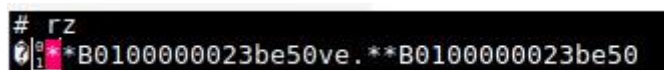
- 1) sz command (send file)
eg: sz filename (the transferred file will be opened under the path of minicom by default).



```
+-----[zmodem download - Press CTRL-C to quit]-----+
|Receiving: cimutils.19
|Bytes received: 41984/ 69020 BPS:10607 ETA 00:02 |
+-----+
|
```

Figure 5-7 Send a screenshot of a file using the lrzsz tool

- 2) rz command (receive file)
➢ First enter rz:



```
# rz
0[*B0100000023be50ve.**B0100000023be50
```

Figure 5-8 The rz command of the lrzsz tool

- Press ctrl+A+Z, then press S (Send file):

```

+-----+
| Minicom Command Summary |
+-----+
| Commands can be called by CTRL-A <key> |
+-----+
| Main Functions           | Other Functions          |
+-----+
| Dialing directory..D    | run script (Go)...G    | Clear Screen.....C |
| Send files.....S      | Receive files.....R    | cOnfigure Minicom..O |
| comm Parameters....P   | Add linefeed.....A    | Suspend minicom....J |
| Capture on/off.....L   | Hangup.....H          | eXit and reset....X |
| send break.....F      | initialize Modem...M   | Quit with no reset.Q |
| Terminal settings..T   | run Kermit.....K      | Cursor key mode...I |
| lineWrap on/off...W    | local Echo on/off..E   | Help screen.....Z   |
| Paste file.....Y      | Timestamp toggle...N   | scroll Back.....B    |
| Add Carriage Ret...U   |                          |                       |
+-----+
| Select function or press Enter for none. |
+-----+

```

Figure 5-9 Screenshot of minicom configuration when using lrzsz tool

- Select zmodem to enter the file selection interface

```

+-----+
| [Select one or more files for upload]-----+
| Directory: /home/al                          |
| [.]                                          |
| [.anaconda]                                |
| [.binwalk]                                 |
| [.cache]                                   |
| [.compiz]                                  |
| [.config]                                  |
| [.crawl]                                   |
| [.dbus]                                    |
| [.ddd]                                     |
| [.debug]                                   |
| [.gconf]                                   |
| [.gnome2]                                  |
+-----+

```

Figure 5-10 Screenshot of the file selection page when using the lrzsz tool

- Up and down keys or PageUp/PageDown to move, space to select files

```

+-----+
| [zmodem upload - Press CTRL-C to quit]-----+
| Sending: main                               |
| Bytes Sent: 9216/ 9976 BPS:671 ETA 00:01 sz: skip |
| ped: main                                  |
| Transfer complete                           |
| READY: press any key to continue...         |
+-----+

```

Figure 5-11 Screenshot of selecting a file when using the lrzsz tool

Note: For more detailed operation parameters, please use (--help) to view.

6 Introduction to Platform Compilation Rules (General)

Compiling different modules on the platform needs to include different core macros, similar to the use of the printf function, the stdio.h header file needs to be included. The core macros used to add different modules are different. The compilation methods of several different types of modules are described below.

6.1 Creating a Platform Application

6.1.1 Compiling core macros

Table 6-1 Platform Application Core Compilation Macros

Compile core macros	illustrate
BUILD_EXECUTABLE	Compile to generate executable bin file
BUILD_SHARED_LIBRARY	Compile and generate dynamic library
BUILD_STATIC_LIBRARY	Compile and generate static library

6.1.2 Commonly used compiler interface macros

Interface macros are used to configure compilation parameters. Commonly used interface macros are shown in the following table.

Table 6-2 Platform application interface macros

interface macro	illustrate
LOCAL_MODULE	module name
LOCAL_MODULE_TAGS	The development mode to which the module belongs
LOCAL_SRC_FILES	Source File
LOCAL_LDLIBS	link parameters
LOCAL_CFLAGS	C compile parameters
LOCAL_CPPFLAGS	CPP compilation parameters
LOCAL_C_INCLUDES	Compile the required header files
LOCAL_MODULE_PATH	Specify the copy path of the target file, do not define this macro to copy the target file to the default path
LOCAL_SHARED_LIBRARIES	Other dynamic libraries that this module depends on for compilation, this dynamic library is generated by other modules in the project
LOCAL_STATIC_LIBRARIES	Other static libraries that this module depends on for compilation, this static library is generated by other modules in the project
LOCAL_DEPANNER_MODULES	Other modules (generated libraries, header files, etc.) that the compilation of this module depends on, here is

the module name of the dependent module.

6.1.3 Compiling the example

1) Executable bin file

This example is a project packages/example/App/cimutils/Build.mk

```
LOCAL_PATH := $(my-dir)

# build cimutils
include $(CLEAR_VARS)
LOCAL_MODULE=cimutils
LOCAL_MODULE_TAGS:= optional

LOCAL_SRC_FILES:= main.c \
    convert_soft.c \
    process_picture/saveraw.c \
    process_picture/savebmp.c \
    process_picture/savejpeg.c \
    lcd/framebuffer.c \
    v4l2/v4l2.c \
    v4l2enc/v4l2_jpegenc.c \
    cim/cim_fmt.c \
    cim/video.c \
    cim/process.c \
    cim/convert.c

LOCAL_LDLIBS := -lc -lstdc++ -ljpeg
LOCAL_C_INCLUDES:= include
LOCAL_DEPANNER_MODULES := CameraDemo.sh

include $(BUILD_EXECUTABLE)
```

2) Generate dynamic library

This example is project external/tinyalsa/Build.mk (for reference only)

```
LOCAL_PATH := $(my-dir):q
include $(CLEAR_VARS)

LOCAL_MODULE := libtinyalsa
LOCAL_MODULE_TAGS := optional
LOCAL_SRC_FILES := pcm.c \
    mixer.c
LOCAL_EXPORT_C_INCLUDE_FILES:=include/tinyalsa/asoundlib.h
```

```
LOCAL_CFLAGS := -Wa,-mips32r2 -O2 -G 0 -Wall -fPIC -shared
```

```
include $(BUILD_SHARED_LIBRARY)
```

Note: The paths of the static library and dynamic library are stored under **usr/lib** of the system by default (the static library in the system is generated from the static library in the sysroot directory, and the static library in the sysroot has not been stripped), and the executable program defaults to Stored in the system's **usr/bin**.

For others, see the build/core/envsetup.mk file under the project or the above-mentioned out directory introduction.

6.2 Importing third-party applications

When adding a third-party module, first ensure that the module can be compiled normally, and then add it to the platform. When adding a third-party function module, it is recommended to place it in the external directory, and refer to each Build.mk file in the external directory to write The Build.mk file of the added module is used to guide the compilation of the added module.

6.2.1 Third-party application core macros

Table 6-3 Third-party application core macros

Compile core macros	illustrate
BUILD_THIRDPART	Compile third-party modules

6.2.2 Common interface macros of third-party applications

Interface macros are used to configure compilation parameters. Commonly used interface macros are shown in the following table.

Table 6-4 Third-party application interface macros

interface macro	illustrate
LOCAL_MODULE	module name
LOCAL_MODULE_TAGS	The development mode to which the module belongs
LOCAL_MODULE_GEN_BINRARY_FILES	Executable program generated by the module (must include path)
LOCAL_MODULE_GEN_STATIC_FILES	The static library file generated by the module (including the path)
LOCAL_MODULE_GEN_SHARED_FILES	Dynamic library file generated by the module (including path)
LOCAL_MODULE_PATH	Specify the copy path of the target file, if this macro is not defined, the target file will be copied

LOCAL_MODULE_CONFIG_FILES	File generated after module configuration
LOCAL_MODULE_CONFIG	How to configure the module
LOCAL_MODULE_COMPILE	How to compile a module
LOCAL_MODULE_COMPILE_CLEAN	module's clean method
LOCAL_EXPORT_C_INCLUDE_FILES	The file exported by the module, the exported file is used by other modules
LOCAL_EXPORT_C_INCLUDE_DIRS	The directory that the module exports to the outside world, and the files in the exported directory are used by other modules
LOCAL_DEPANNER_MODULES	Other modules (generated libraries, header files, etc.) that the compilation of this module depends on, here is the module name of the dependent module

6.2.3 Compiling the example

This example is a project packages/example/App/linphone-desktop/Build.mk

```
LOCAL_PATH := $(my-dir)
include $(CLEAR_VARS)

LOCAL_MODULE=linphone-desktop
LOCAL_DEPANNER_MODULES:=isp
LOCAL_MODULE_TAGS:=optional
LOCAL_MODULE_GEN_BINRARY_FILES=OUTPUT/no-ui/bin/linphonec
LOCAL_MODULE_GEN_SHARED_FILES=OUTPUT/no-ui/lib/libavcodec.so
                                OUTPUT/no-ui/lib/libavcodec.so.53 \
                                .....
                                OUTPUT/no-ui/lib/libxml2.so.2
LOCAL_MODULE_CONFIG_FILES:= Makefile
LOCAL_MODULE_CONFIG:=./prepare.py \
                                -L -dv \
                                .....
                                -DENABLE_INGENIC_VIDEOOUT=ON
LOCAL_MODULE_COMPILE=make -j$(MAKE_JLEVEL); mkdir -p
                                $(TOP_DIR)/$(LOCAL_PATH)/OUTPUT/no-ui/data/linphone
LOCAL_MODULE_COMPILE_CLEAN=./prepare.py -c

include $(BUILD_THIRDPART)
```

6.3 Add a module for copying files

In the development process, it is often necessary to copy some libraries and header files from one path to another target path without any modification. A prebuild mechanism is added to the compilation system of this platform.

6.3.1 Compiling core macros

Table 6-5 Copy module core macro

Compile core macros	illustrate
BUILD_PREBUILT	copy one file at a time
BUILD_MULTI_COPY	Copy multiple files at once

6.3.2 Common interface macros

Interface macros are used to configure compilation parameters. Commonly used interface macros are shown in the following table.

Table 6-6 Copy module interface macro

interface macro	illustrate
LOCAL_MODULE	module name
LOCAL_MODULE_TAGS	The development mode to which the module belongs
LOCAL_MODULE_CLASS	Which prebuild mode DIR/FILES the module belongs to
LOCAL_MODULE_PATH	The target path of the file (folder) copy
LOCAL_COPY_FILES	Source File
LOCAL_MODULE_DIR	source folder

6.3.3 Compiling the example

1) Copy a single file

This example is a project packages/example/App/cimutils/Build.mk

```
# copy the LOCAL_SRC_FILES

include $(CLEAR_VARS)
LOCAL_MODULE := CameraDemo.sh
LOCAL_MODULE_TAGS := optional
LOCAL_MODULE_PATH := $(TARGET_FS_BUILD)/$(TARGET_TESTSUITE_DIR)/CameraDemo
LOCAL_COPY_FILES := CameraDemo.sh:CameraDemo.sh

include $(BUILD_PREBUILT)
```

The macro LOCAL_COPY_FILES := CameraDemo.sh:CameraDemo.sh, separated by ":", the front is the copied name (which can contain the path, and must have the target file name), and the back is the local file name (including the path).

2) Copy multiple files

This example is a project external/android/include/Build.mk

```

LOCAL_PATH := $(my-dir)
# copy include

include $(CLEAR_VARS)

LOCAL_MODULE := android-include
LOCAL_MODULE_TAGS := optional
LOCAL_MODULE_PATH := $(OUT_DEVICE_INCLUDE_DIR)
LOCAL_MODULE_CLASS := DIR
LOCAL_MODULE_DIR := android-include

include $(BUILD_MULTI_COPY)

```

6.4 Automatically generate modules by command

After creating the module directory, run autotouchmk in this directory, and then select the type of module to be added (including executable module, third-party module, copy module, static library module, dynamic library module).

After running the autotouchmk command, there will be the following prompts:

```

This command is used to generate the commonly used template, specific content need to edit
Host module or device module(A:host   B:device):
Input Module :

```

Select the module type, controller module: input A; equipment module: input B; the following prompt will be displayed after input:

```

This command is used to generate the commonly used template, specific content need to edit
Host module or device module(A:host   B:device):
Input Module :A
Please select host target type you want to build
(A:execute   B:thirdpart   C:static library   D:shared library):
Host

```

```

This command is used to generate the commonly used template, specific content need to edit
Host module or device module(A:host   B:device):
Input Module :B

*****

Please select device target type you want to build
A:execute
B:thirdpart

```

```

C:static library
D:shared library
E:multi prebuilt
F:multi copy
Input module :

```

Executable module: input A; third-party module: input B; static library module: input C; dynamic library module: input D; copy module: input E; multiple copy module: input F;

Then a Build.mk file will be generated in the current directory. The automatically generated Build.mk will list all the macros that will be used basically, and assign values to the macros according to your needs. If you don't need them, you can leave them unmodified. The automatically generated Build.mk file has no assignment to most macros, and users need to assign values according to their own needs.

6.4.1 Example of generating a copy module

Execute autotouchmk in the directory where the compiled module needs to be created, and the following prompt message will appear.

```

This command is used to generate the commonly used template, specific content need to edit
Host module or device module(A:host   B:device):
Input Module :B

*****
Please select device target type you want to build
A:execute
B:thirdpart
C:static library
D:shared library
E:multi prebuilt
F:multi copy
Input module :

```

We select option B, select the device module, and then select option E to create a copy module. The generated content is as follows.

```

LOCAL_PATH := $(my-dir)
include $(CLEAR_VARS)
LOCAL_MODULE =      #name of the module:
LOCAL_MODULE_TAGS := optional      #Development mode of module ,userdebug,eng,optional
LOCAL_MODULE_PATH := $(TARGET_FS_BUILD)/      #which directory the target file copy
LOCAL_MODULE_CLASS:=
LOCAL_MODULE_DIR :=      #src module dir
LOCAL_COPY_FILES:=
include $(BUILD_PREBUILT)

```

6.5 File system overlay rootfs-overlay

halley6 platform provides a rootfs-overlay mechanism for modifying the content in the generated file system. rootfs-overlay is a folder located under the device/(halley6)board/ directory. When there is a file with the same file name in the file system in the folder, the file in the file system will be replaced with the file in the current directory.

6.6 Making the root file system

The halley6 platform supports the creation of various types of file systems. You can create different file systems by configuring the parameters of the file system. The following are several specific file system creation methods.

6.6.1 jffs2 file system creation

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Filesystem images/jffs2 root filesystem/

The following figure shows the reference configuration of the jffs2 file system.

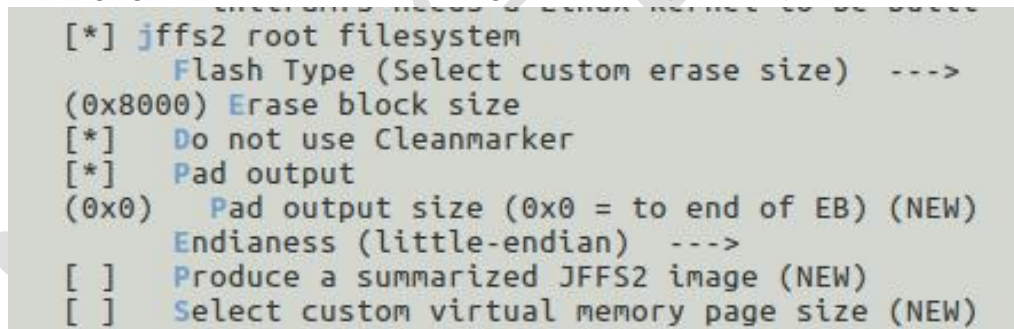


Figure 6-1 Reference configuration of jffs2 file system

After the configuration is complete, execute the make command in the project directory to compile, and the generated files will be stored in the **out/product/halley6/image/** directory.

6.6.2 ubifs file system creation

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Filesystem images/ubifs root filesystem/

The following figure shows the reference configuration of the ubifs file system.

```
[*] ubifs root filesystem
(0x1f000) logical eraseblock size
(0x800) minimum I/O unit size
(2048) maximum logical eraseblock count
ubifs runtime compression (lzo) --->
Compression method (no compression) --->
() Additional mkfs.ubifs options
```

Figure 6-2 Reference configuration of ubifs file system

After the configuration is complete, execute the make command in the project directory to compile, and the compiled files will be stored in the **out/product/halley6/image/** directory.

6.6.3 ext4 file system creation

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Filesystem images/ext2/3/4 root filesystem/

The following is the reference configuration of the ext4 file system.

```
[*] ext2/3/4 root filesystem
ext2/3/4 variant (ext2 (rev1)) --->
() filesystem label
(120M) exact size
(0) exact number of inodes (leave at 0 for auto calculation)
(5) reserved blocks percentage
(-0 ^64bit) additional mke2fs options
Compression method (no compression) --->
```

Figure 6-3 Reference configuration of the ext4 file system

After the configuration is complete, execute the make command in the project directory to compile, and the compiled files will be stored in the **out/product/halley6/image/** directory.

7 Apps debug

7.1 Debug using the gdb command line tool

7.1.1 gdb tool configuration

- 1) Enter halley6 project
- 2) Execute make buildroot-menuconfig
- 3) Configuration Target packages/Debugging, profiling and benchmark/gdb, As shown below.

```
Symbol: BR2_PACKAGE_GDB [=n]
Type : bool
Prompt: gdb
Location:
  -> Target packages
  -> Debugging, profiling and benchmark
Defined at package/gdb/Config.in:17
Depends on: BR2_TOOLCHAIN_HAS_THREADS [=y] && BR2_TOOLCHAIN_HAS_THREADS_DEBUG [=y]
BR2_TOOLCHAIN_GCC_AT_LEAST_4_8 [=y] && BR2_INSTALL_LIBSTDCPP [=y]
Selects: BR2_PACKAGE_GDB_DEBUGGER [=n] && BR2_PACKAGE_GDB_SERVER [=n]
```

Figure 7-1 gdb configuration path in buildroot

7.1.2 Gdb local debugging

To debug with gdb, you need to add the -g option when compiling the executable program. Then execute the following command to enter gdb debug mode.

gdb executable file name.

The debugging interface is shown in the following figure.

```

zhxiao@SW-PowerEdge-R610:~$
zhxiao@SW-PowerEdge-R610:~$ gdb test
GNU gdb (Ubuntu 7.7-0ubuntu3) 7.7
Copyright (C) 2014 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test...done.
(gdb)
(gdb)
(gdb) list
9             int c = 0;
10
11             c = a + b;
12
13             return c;
14         }
15
16
17     int main(void)
18     {
(gdb)
19         int sum;
20         int i;
21
22         for(i = 0; i < 10; i++)
23         {
24             sum += add(i,i);
25         }
26
27         printf("sum = %d\n",sum);
28
(gdb)
29         return 0;
30     }

```

Figure 7-2 gdb local debugging interface

7.1.3 gdbserver + gdb Remote debugging

In many cases, it is necessary to repeatedly debug an application, especially complex programs. The GDB method is used to debug. Due to the limited resources of the embedded system, it is generally not possible to debug directly on the target system. Usually, the method of gdb+gdbserver is used to debug. gdbserver runs on the target system and gdb runs on the host.

7.1.3.1 Installation of gdbserver

gdbserver can be generated using source code compilation, and the source code of gdb is provided in halley6. In the buildroot/dl/gdb/ directory.

halley6 also provides compiled gdbserver, users only need to copy the gdbserver file to the board,

the path is as follows:

prebuilts/toolchains/mips-gcc720-glibc229/mips-linux-gnu/libc/mfp64/usr/lib/bin

7.1.3.2 gdbserver + gdb Debug method

Execute the following command on the target board:

```
gdbserver Host IP: Port number Executable program to be debugged
```

The port number can be freely set.

Execute the following command on the host:

```
mips-linux-gnu-gdb executable program to debug
```

Then it will enter the gdb debugging interface, and then execute the command

```
target remote Target Board IP:Port Number
```

The executable program must be the same as the one running on the target board, and the port number must be the same.

The following figure is a screenshot of the debugging process using gdbserver + gdb debugging.

```
# ./gdbserver 192.168.4.50:1234 hello
Process hello created; pid = 5343
Listening on port 1234
```

Figure 7-3 The target board waits for connection after executing the command

```
user@tftpboot$ mips-linux-gnu-gdb hello
GNU gdb (Ingenic r4.1.0-gcc720 2018.08-06) 7.11.50.20160803-git
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=i686-pc-linux-gnu --target=mips-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from hello...done.
(gdb)
(gdb) target remote 192.168.4.49:1234
Remote debugging using 192.168.4.49:1234
0x004002b0 in __start ()
(gdb)
```

Figure 7-4 The host computer enters the gdb debugging interface and connects to the target board

```
# ./gdbserver 192.168.4.50:1234 hello
Process hello created; pid = 5343
Listening on port 1234
Remote debugging from host 192.168.4.50
```


Figure 7-5 The connection between the target board and the host computer is successfully established

```
(gdb)
(gdb) target remote 192.168.4.49:1234
Remote debugging using 192.168.4.49:1234
0x004002b0 in __start ()
(gdb)
(gdb) list
1      #include <stdio.h>
2
3
4
5      int main(void)
6      {
7          int i;
8
9          for(i = 0; i < 10; i++)
10         {
(gdb) list
11             printf("[%d] hello world!\n",i);
12         }
13
14         printf("hello world test over!\n");
15     }
16
17
18
19
20
(gdb) b main
Breakpoint 1 at 0x40024c: file helloworld.c, line 6.
(gdb) c
Continuing.

Breakpoint 1, main () at helloworld.c:6
6      {
(gdb) r
The "remote" target does not support "run". Try "help target" or "continue".
(gdb) n
Inferior 1 (process 5343) exited normally]
(gdb) n
The program is not being run.
(gdb) █
```

Figure 7-6 Host computer to debug

```
# ./gdbserver 192.168.4.50:1234 hello
Process hello created; pid = 5343
Listening on port 1234
Remote debugging from host 192.168.4.50
[0] hello world!
[1] hello world!
[2] hello world!
[3] hello world!
[4] hello world!
[5] hello world!
[6] hello world!
[7] hello world!
[8] hello world!
[9] hello world!
hello world test over!

Child exited with status 0
# █
```

Figure 7-7 The target board prints debugging information

7.1.4 gdb Debug using the command

1) Run the command

run: Abbreviated as *r*, its function is to run the program. When a breakpoint is encountered, the program will stop running at the breakpoint and wait for the user to enter the next command.

continue: (abbreviated *c*): continue execution to the next breakpoint (or end of run)

next: (Abbreviation *n*), single-step tracking program, when encountering a function call, it does not enter the function body; the main difference between this command and step is that step encounters a user-defined function, it will step into the function to run, And next directly calls the function without entering the function body.

Step: (abbreviated *s*): single-step debugging if there is a function call, enter the function; different from the command *n*, *n* does not enter the called function

until: When you're tired of stepping through a loop body, this command runs the program until it exits the loop body

until+ line number: run to a line, not just to break out of a loop

finish: Run the program until the current function returns, and print the stack address, return value, parameter value and other information when the function returns

call Function (parameter): Call a function visible in the program, and pass "parameters", such as: `call gdb_test(55)`

quit: Abbreviated as *q*, exit gdb

2) Set breakpoints

break n (abbreviated *b n*): set a breakpoint at line *n*

b fn1 if a > b: Conditional breakpoint settings

break func (break is abbreviated as *b*): set a breakpoint at the entry of the function `func()`

delete Breakpoint number n: delete the *n*th breakpoint

disable Breakpoint number n: Pause at the *n*th breakpoint

enable Breakpoint number n: open the *n*th breakpoint

clear line number n: clear the breakpoint on line *n*

info b (info breakpoints) :Display the breakpoint settings of the current program

delete breakpoints: clear all breakpoints

3) View the source code

list :It is abbreviated as *l*, and its function is to list the source code of the program. By default, 10 lines are displayed each time.

list Line number: will display the 10 lines of code before and after the current file centered on the "line number"

list Function name: will display the source code of the function where the "function name" is located, such as: `list main`

list :Without parameters, it will follow the previous list command and output the following content.

4) print expression

print Expression: abbreviated as p, where "expression" can be any valid expression of the program currently being tested, such as a program currently being debugged in C language, then "expression" can be any valid expression in C language, Including numbers, variables and even function calls.

print a: will display the value of the integer a

print ++a: will add 1 to the value in a and display it

print name: will display the value of the string name

print gdb_test(22): will call the gdb_test() function with the integer 22 as an argument

print gdb_test(a): will call the gdb_test() function with the variable a as an argument

display Expression: It is very useful in single-step operation. After using the display command to set an expression, it will output the set expression and value immediately after each single-step instruction. Such as: display a

watch Expression: Set a watchpoint, once the value of the watched "expression" changes, gdb will forcibly terminate the program being debugged. Such as: watch a

whatis :query variable or function

info function: query function

Extended info locals: Displays all variables of the current stack page

5) View running information

where/bt :List of currently running stacks;

bt backtrace Display the current call stack

up/down Change the depth of the stack display

set args Parameters: Specify the parameters at runtime

show args: Check the set parameters

info program: Check whether the program is running, the process number, and the reason for being suspended..

6) Split window

layout: Used to split the window, you can view the code while testing

layout src: Show source code window

layout asm: show disassembly window

layout regs: Show source code/disassembly and CPU registers window

layout split: Show source code and disassembly window

Ctrl + L: refresh window

7.2 Debug using the vscode integrated development environment

7.2.1 vscode installation and use

vscode Download the official installation package: [<https://code.visualstudio.com/Download>]

vscode Official installation reference documentation:
[<https://code.visualstudio.com/docs/setup/linux>]

vscode Official debugging reference documentation:
[<https://code.visualstudio.com/docs/cpp/cpp-debug>]

vscode Official debug configuration reference documentation:
[<https://code.visualstudio.com/docs/cpp/launch-json-reference>]

7.2.2 Debugging Environment Construction and Project Compilation

In the top-level directory of the halley6 project, you can configure the project cross-debugging through the following commands

```
make <ProjectName>-vscode PRO=<ProjectName>
```

The <ProjectName> can be any name. For the convenience of memory, the board-level name is generally used as the ProjectName.

After the command is executed, a <ProjectName>.code-workspace will be generated in the current directory.

After entering the project directory, execute the following command to enter the workspace (please confirm that vscode has been successfully installed before executing the command).

```
code <ProjectName>.code-workspace
```

After the command is executed, it will enter the visual debugging stage of vscode.

On the source file interface to be compiled, click the green "△" icon in the upper left, or click Run in the menu bar and select StartDebugging to compile and debug the file. As shown below.

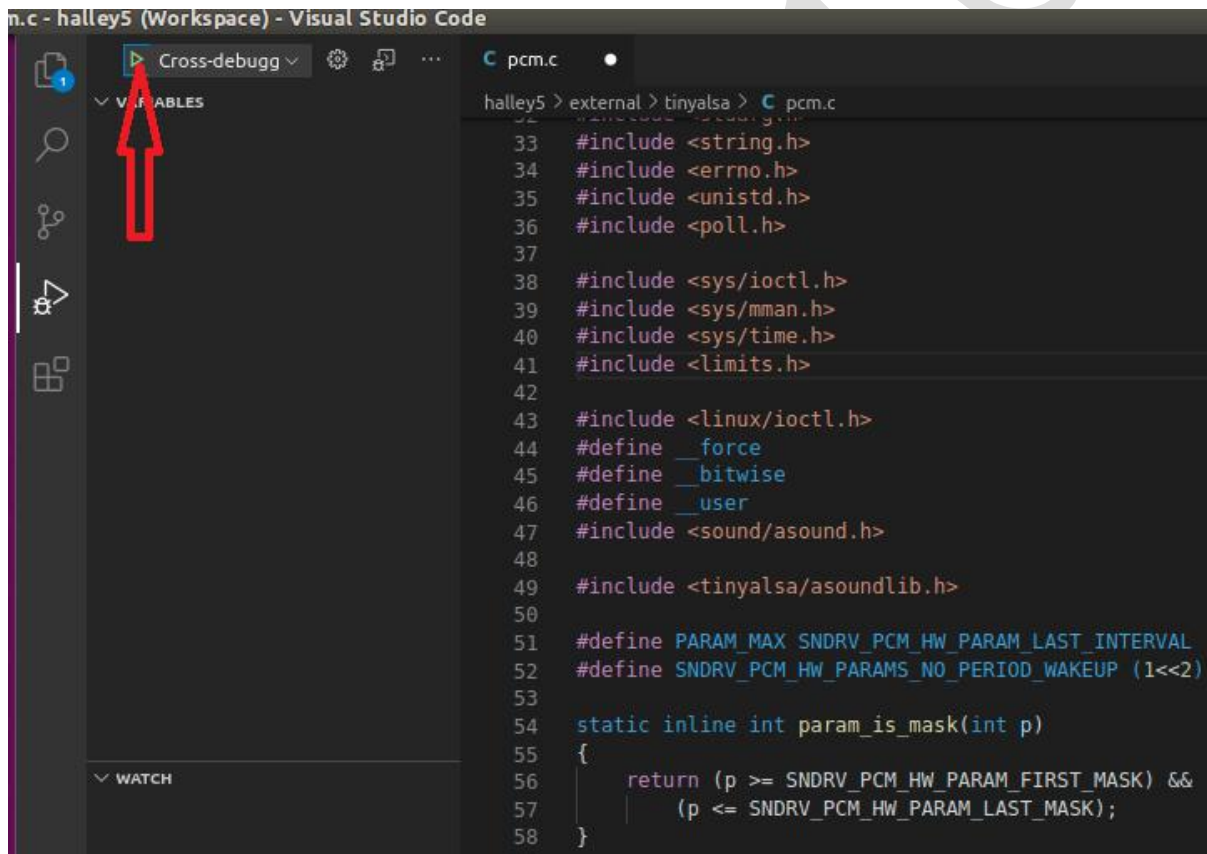


Figure 7-8 Screenshot of vscode interface

In the top-level directory of halley6 software project, the following commands can be used to clear the project configuration information

```
make <ProjectName>-clean PRO=<ProjectName>
```

7.2.3 Start debugging

F5 starts debugging, you can set breakpoints through F9, single step through F11, etc. to debug

the program.

ingenic